LABSHEET - 9

HIMANI CHAUDHARY

BTECH(CSE) 3RD YEAR

CU240250967

SECTION - B

ROLL NO. - 24

Mastery in Data Analytics and Visualizations and Career Advancement (Technical)

```
# ============================================================
# EXPERIMENT NO. 10
# END-TO-END DATA ANALYTICS PROJECT
# CUSTOMER CHURN ANALYSIS
# ============================================================


# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import chi2_contingency

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score
)

import warnings
warnings.filterwarnings("ignore")
print("Himani Chaudhary")
print("CU240250967")
```

```
Himani Chaudhary
CU240250967
```

```
# 2. LOAD DATASET
# ============================================================

# IMPORTANT:
# Use r before the path to avoid Windows backslash errors.

file_path ="/content/churn_prediction_results.csv"

print("\n============================================================")
print("EXPERIMENT NO. 10")
print("END-TO-END DATA ANALYTICS PROJECT")
print("CUSTOMER CHURN ANALYSIS")
print("============================================================")

print("\nLoading dataset...")

try:

    df = pd.read_csv(file_path)

    print("\nDataset loaded successfully!")

except FileNotFoundError:

    print("\nERROR: Dataset file was not found.")
    print("\nPlease check this path:")
    print(file_path)

    raise SystemExit
```

```
except Exception as e:

    print("\nERROR while loading dataset:")
    print(e)

    raise SystemExit
    print("Himani Chaudhary")
    print("CU240250967")
```

```
============================================================
EXPERIMENT NO. 10
END-TO-END DATA ANALYTICS PROJECT
CUSTOMER CHURN ANALYSIS
============================================================

Loading dataset...

Dataset loaded successfully!
```

```
# 3. DISPLAY FIRST FIVE ROWS
# ============================================================

print("\n============================================================")
print("FIRST FIVE ROWS")
print("============================================================")

print(df.head())
print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
FIRST FIVE ROWS
============================================================
   Actual_Churn  Predicted_Churn
0         False            False
1         False             True
2         False            False
3          True            False
4         False            False
Himani Chaudhary
CU240250967
```

```
# 4. DATASET INFORMATION
# ============================================================

print("\n============================================================")
print("DATASET INFORMATION")
print("============================================================")

print("\nNumber of rows and columns:")
print(df.shape)

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)
print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
DATASET INFORMATION
============================================================

Number of rows and columns:
(1407, 2)

Column names:
['Actual_Churn', 'Predicted_Churn']

Data types:
Actual_Churn       bool
Predicted_Churn    bool
dtype: object
Himani Chaudhary
CU240250967
```

```
# 5. CHECK MISSING VALUES
# ============================================================

print("\n============================================================")
print("MISSING VALUES")
print("============================================================")

missing_values = df.isnull().sum()

print(missing_values)

total_missing = missing_values.sum()

if total_missing == 0:

    print("\nNo missing values found.")

else:

    print("\nMissing values found.")
    print("Total missing values:", total_missing)
print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
MISSING VALUES
============================================================
Actual_Churn       0
Predicted_Churn    0
dtype: int64

No missing values found.
Himani Chaudhary
CU240250967
```

```
# 6. HANDLE MISSING VALUES
# ============================================================

print("\n============================================================")
print("HANDLING MISSING VALUES")
print("============================================================")

for column in df.columns:

    if df[column].isnull().sum() > 0:

        # Categorical column
        if df[column].dtype == "object":

            mode_value = df[column].mode()

            if len(mode_value) > 0:

                df[column] = df[column].fillna(
                    mode_value[0]
                )

        # Numeric column
        else:

            median_value = df[column].median()

            df[column] = df[column].fillna(
                median_value
            )


print("Missing values handled successfully.")

print("\nMissing values after cleaning:")
print(df.isnull().sum())
print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
HANDLING MISSING VALUES
============================================================
Missing values handled successfully.
```

```
Missing values after cleaning:
Actual_Churn       0
Predicted_Churn    0
dtype: int64
Himani Chaudhary
CU240250967
```

```
# 7. CHECK DUPLICATE VALUES
# ============================================================

print("\n============================================================")
print("DUPLICATE VALUES")
print("============================================================")

duplicate_count = df.duplicated().sum()

print(
    "Number of duplicate rows:",
    duplicate_count
)

if duplicate_count > 0:

    df = df.drop_duplicates()

    print("Duplicate rows removed.")

else:

    print("No duplicate rows found.")
print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
DUPLICATE VALUES
============================================================
Number of duplicate rows: 1403
Duplicate rows removed.
Himani Chaudhary
CU240250967
```

```
# 8. DESCRIPTIVE STATISTICS
# ============================================================

print("\n============================================================")
print("DESCRIPTIVE STATISTICS")
print("============================================================")

print(
    df.describe(include="all")
)
print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
DESCRIPTIVE STATISTICS
============================================================
       Actual_Churn Predicted_Churn
count             4               4
unique            2               2
top           False           False
freq              2               2
Himani Chaudhary
CU240250967
```

```
#9. IDENTIFY NUMERIC AND CATEGORICAL COLUMNS
# ============================================================

numeric_columns = df.select_dtypes(
    include=np.number
).columns.tolist()

categorical_columns = df.select_dtypes(
    include="object"
).columns.tolist()


print("\n============================================================")
print("NUMERIC COLUMNS")
print("============================================================")
```

```
print(numeric_columns)


print("\n============================================================")
print("CATEGORICAL COLUMNS")
print("============================================================")

print(categorical_columns)
print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
NUMERIC COLUMNS
============================================================
[]

============================================================
CATEGORICAL COLUMNS
============================================================
[]
Himani Chaudhary
CU240250967
```

```
# 10. FIND ACTUAL CHURN COLUMN
# ============================================================

actual_churn_column = None

actual_candidates = [
    "Actual_Churn",
    "Actual Churn",
    "ActualChurn",
    "actual_churn",
    "actual",
    "Churn",
    "churn"
]


# First search using exact candidate names

for column in actual_candidates:

    if column in df.columns:

        actual_churn_column = column

        break


# Automatic search

if actual_churn_column is None:

    for column in df.columns:

        column_name = column.lower()

        if (
            "churn" in column_name
            and "predict" not in column_name
            and "prob" not in column_name
        ):

            actual_churn_column = column

            break
print("Himani Chaudhary")
print("CU240250967")
```

```
Himani Chaudhary
CU240250967
```

```
# 11. FIND PREDICTED CHURN COLUMN
# ============================================================

predicted_churn_column = None
```

```
predicted_candidates = [
    "Predicted_Churn",
    "Predicted Churn",
    "PredictedChurn",
    "predicted_churn",
    "prediction",
    "Prediction",
    "Predicted",
    "predicted"
]


# Exact candidate search

for column in predicted_candidates:

    if column in df.columns:

        predicted_churn_column = column

        break


# Automatic search

if predicted_churn_column is None:

    for column in df.columns:

        column_name = column.lower()

        if (
            "predict" in column_name
            and "prob" not in column_name
        ):

            predicted_churn_column = column

            break
print("Himani Chaudhary")
print("CU240250967")
```

```
Himani Chaudhary
CU240250967
```

```
# 12. FIND CHURN PROBABILITY COLUMN
# ============================================================

probability_column = None

probability_candidates = [
    "Churn_Probability",
    "Churn Probability",
    "ChurnProbability",
    "Probability",
    "probability",
    "Predicted_Probability",
    "Predicted Probability",
    "churn_probability"
]


# Exact candidate search

for column in probability_candidates:

    if column in df.columns:

        probability_column = column

        break


# Automatic search

if probability_column is None:

    for column in df.columns:

        column_name = column.lower()
```

```
        if (
            "prob" in column_name
            and "churn" in column_name
        ):

            probability_column = column

            break
print("Himani Chaudhary")
print("CU240250967")
```

```
Himani Chaudhary
CU240250967
```

```
# 13. DISPLAY DETECTED CHURN COLUMNS
# ============================================================

print("\n============================================================")
print("CHURN RELATED COLUMNS")
print("============================================================")

print(
    "Actual Churn Column:",
    actual_churn_column
)

print(
    "Predicted Churn Column:",
    predicted_churn_column
)

print(
    "Probability Column:",
    probability_column
)
print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
CHURN RELATED COLUMNS
============================================================
Actual Churn Column: Actual_Churn
Predicted Churn Column: Predicted_Churn
Probability Column: None
Himani Chaudhary
CU240250967
```

```
# 14. ACTUAL CHURN DISTRIBUTION
# ============================================================

if actual_churn_column is not None:

    print("\n============================================================")
    print("ACTUAL CHURN DISTRIBUTION")
    print("============================================================")

    print(
        df[actual_churn_column].value_counts()
    )

    plt.figure(figsize=(8, 5))

    sns.countplot(
        x=df[actual_churn_column]
    )

    plt.title(
        "Actual Customer Churn Distribution"
    )

    plt.xlabel("Churn")
    plt.ylabel("Number of Customers")

    plt.tight_layout()

    plt.show()


else:
```

```
    print("\nActual churn column was not found.")


# ============================================================
# 15. PREDICTED CHURN DISTRIBUTION
# ============================================================

if predicted_churn_column is not None:

    print("\n============================================================")
    print("PREDICTED CHURN DISTRIBUTION")
    print("============================================================")

    print(
        df[predicted_churn_column].value_counts()
    )

    plt.figure(figsize=(8, 5))

    sns.countplot(
        x=df[predicted_churn_column]
    )

    plt.title(
        "Predicted Customer Churn Distribution"
    )

    plt.xlabel("Predicted Churn")
    plt.ylabel("Number of Customers")

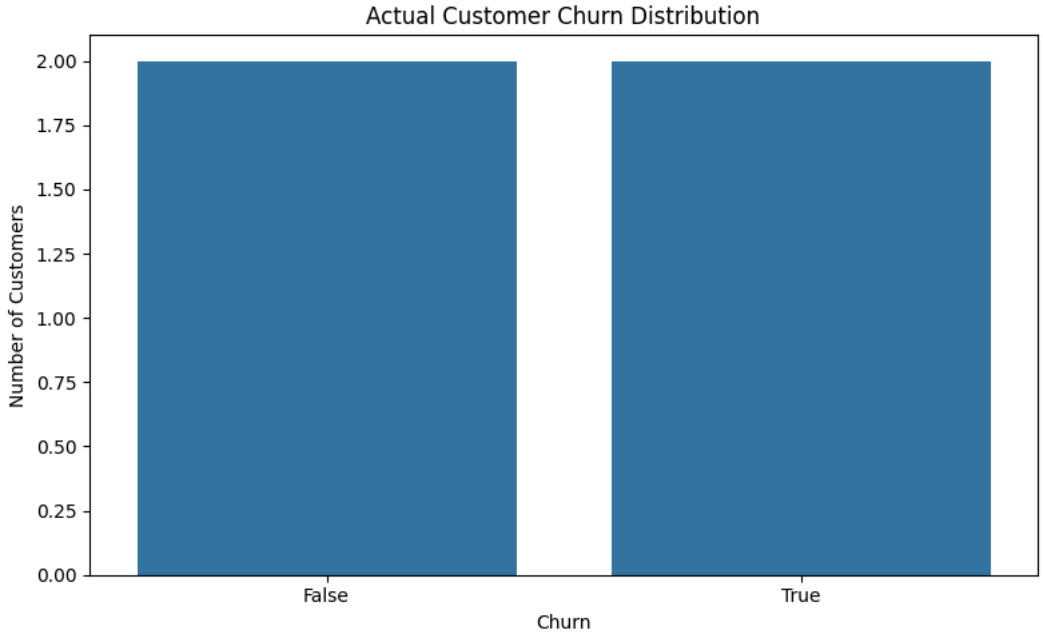
    plt.tight_layout()

    plt.show()

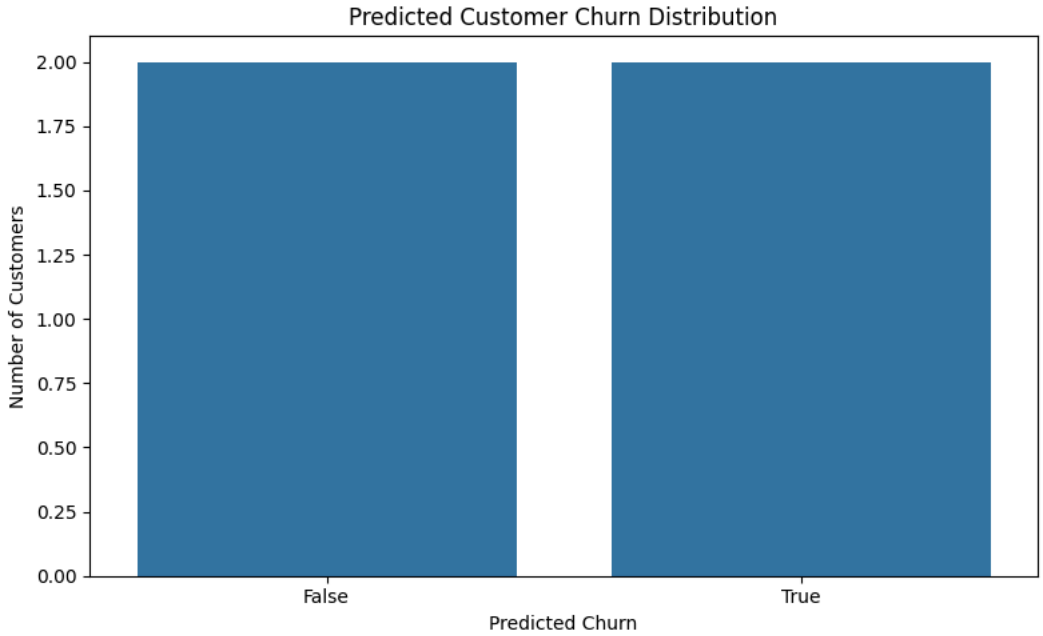

else:

    print("\nPredicted churn column was not found.")
print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
ACTUAL CHURN DISTRIBUTION
============================================================
Actual_Churn
False    2
True     2
Name: count, dtype: int64
```




```
============================================================
PREDICTED CHURN DISTRIBUTION
============================================================
Predicted_Churn
False    2
True     2
Name: count, dtype: int64
```




```
Himani Chaudhary
CU240250967
```

```
# 16. CHURN PROBABILITY ANALYSIS
# ============================================================

if probability_column is not None:

    print("\n============================================================")
    print("CHURN PROBABILITY ANALYSIS")
    print("============================================================")

    probability_values = pd.to_numeric(
        df[probability_column],
        errors="coerce"
```

```
    )

    print(
        probability_values.describe()
    )

    plt.figure(figsize=(8, 5))

    sns.histplot(
        probability_values.dropna(),
        bins=20,
        kde=True
    )

    plt.title(
        "Churn Probability Distribution"
    )

    plt.xlabel("Churn Probability")
    plt.ylabel("Number of Customers")

    plt.tight_layout()

    plt.show()


else:

    print("\nChurn probability column was not found.")
    print("Himani Chaudhary")
print("CU240250967")
```

```
Churn probability column was not found.
Himani Chaudhary
CU240250967
```

```
# 17. HISTOGRAMS FOR NUMERIC VARIABLES
# ============================================================

print("\n============================================================")
print("NUMERIC VARIABLE DISTRIBUTIONS")
print("============================================================")

if len(numeric_columns) > 0:

    for column in numeric_columns:

        plt.figure(figsize=(8, 5))

        sns.histplot(
            df[column].dropna(),
            bins=20,
            kde=True
        )

        plt.title(
            "Distribution of " + column
        )

        plt.xlabel(column)
        plt.ylabel("Frequency")

        plt.tight_layout()

        plt.show()

else:

    print("No numeric columns found.")
    print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
NUMERIC VARIABLE DISTRIBUTIONS
============================================================
No numeric columns found.
Himani Chaudhary
CU240250967
```

```
# 18. BOX PLOTS
# ============================================================

print("\n============================================================")
print("BOX PLOT ANALYSIS")
print("============================================================")

if len(numeric_columns) > 0:

    for column in numeric_columns:

        plt.figure(figsize=(8, 5))

        sns.boxplot(
            y=df[column]
        )

        plt.title(
            "Box Plot of " + column
        )

        plt.ylabel(column)

        plt.tight_layout()

        plt.show()

else:

    print("No numeric columns found.")
    print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
BOX PLOT ANALYSIS
============================================================
No numeric columns found.
Himani Chaudhary
CU240250967
```

```
# 19. CORRELATION MATRIX
# ============================================================

if len(numeric_columns) >= 2:

    print("\n============================================================")
    print("CORRELATION MATRIX")
    print("============================================================")

    correlation = df[numeric_columns].corr()

    print(correlation)

    plt.figure(figsize=(10, 7))

    sns.heatmap(
        correlation,
        annot=True,
        cmap="coolwarm",
        fmt=".2f"
    )

    plt.title(
        "Correlation Heatmap"
    )

    plt.tight_layout()

    plt.show()


else:

    print("\nNot enough numeric columns for correlation analysis.")
    print("Himani Chaudhary")
print("CU240250967")
```

```
Not enough numeric columns for correlation analysis.
```

```
Himani Chaudhary
CU240250967
```

```
# 20. CHI-SQUARE TEST
# ============================================================

if actual_churn_column is not None:

    suitable_category = None

    for column in categorical_columns:

        if column != actual_churn_column:

            if df[column].nunique() <= 10:

                suitable_category = column

                break


    if suitable_category is not None:

        print("\n============================================================")
        print("CHI-SQUARE TEST")
        print("============================================================")

        print(
            "Testing relationship between:",
            suitable_category,
            "and",
            actual_churn_column
        )

        contingency_table = pd.crosstab(
            df[suitable_category],
            df[actual_churn_column]
        )

        print("\nContingency Table:")

        print(contingency_table)


        try:

            chi2, p_value, dof, expected = chi2_contingency(
                contingency_table
            )

            print(
                "\nChi-Square Value:",
                chi2
            )

            print(
                "P-Value:",
                p_value
            )

            print(
                "Degrees of Freedom:",
                dof
            )


            if p_value < 0.05:

                print(
                    "Result: Significant relationship exists."
                )

            else:

                print(
                    "Result: No significant relationship found."
                )


        except Exception as e:

            print(
```

```
                "Chi-square test could not be performed."
            )

            print(e)

    else:

        print(
            "\nNo suitable categorical column found for Chi-square test."
        )
        print("Himani Chaudhary")
print("CU240250967")
```

```
No suitable categorical column found for Chi-square test.
Himani Chaudhary
CU240250967
```

```
# 21. CONVERT CHURN VALUES TO NUMERIC
# ============================================================

def convert_churn_values(series):

    mapping = {

        "yes": 1,
        "no": 0,

        "y": 1,
        "n": 0,

        "true": 1,
        "false": 0,

        "churn": 1,
        "no churn": 0,

        "churned": 1,
        "not churned": 0,

        "1": 1,
        "0": 0
    }


    result = []


    for value in series:

        text_value = str(value).strip().lower()


        if text_value in mapping:

            result.append(
                mapping[text_value]
            )


        else:

            try:

                number = float(value)


                if number in [0, 1]:

                    result.append(
                        int(number)
                    )

                else:

                    result.append(
                        np.nan
                    )


            except:
```

```
                result.append(
                    np.nan
                )


    return pd.Series(
        result,
        index=series.index,
        dtype="float"
    )
print("Himani Chaudhary")
print("CU240250967")
```

```
Himani Chaudhary
CU240250967
```

```
# 22. MODEL EVALUATION
# ============================================================

actual_numeric = None
predicted_numeric = None

y_actual = pd.Series(
    dtype=float
)

y_predicted = pd.Series(
    dtype=float
)


if (
    actual_churn_column is not None
    and predicted_churn_column is not None
):

    actual_numeric = convert_churn_values(
        df[actual_churn_column]
    )

    predicted_numeric = convert_churn_values(
        df[predicted_churn_column]
    )


    valid_rows = (
        actual_numeric.notna()
        &
        predicted_numeric.notna()
    )


    y_actual = actual_numeric[
        valid_rows
    ]

    y_predicted = predicted_numeric[
        valid_rows
    ]


    print("\n============================================================")
    print("MODEL PERFORMANCE EVALUATION")
    print("============================================================")


    if len(y_actual) > 0:

        accuracy = accuracy_score(
            y_actual,
            y_predicted
        )


        precision = precision_score(
            y_actual,
            y_predicted,
            zero_division=0
        )


        recall = recall_score(
```

```
            y_actual,
            y_predicted,
            zero_division=0
        )


        f1 = f1_score(
            y_actual,
            y_predicted,
            zero_division=0
        )


        print(
            "\nAccuracy :",
            round(accuracy, 4)
        )

        print(
            "Precision:",
            round(precision, 4)
        )

        print(
            "Recall   :",
            round(recall, 4)
        )

        print(
            "F1 Score :",
            round(f1, 4)
        )


    else:

        print(
            "No valid actual/predicted churn values found."
        )


else:

    print(
        "\nActual or predicted churn column was not found."
    )
print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
MODEL PERFORMANCE EVALUATION
============================================================

Accuracy : 0.5
Precision: 0.5
Recall   : 0.5
F1 Score : 0.5
Himani Chaudhary
CU240250967
```

```
# 23. CONFUSION MATRIX
# ============================================================

if len(y_actual) > 0:

    cm = confusion_matrix(
        y_actual,
        y_predicted
    )


    print("\n============================================================")
    print("CONFUSION MATRIX")
    print("============================================================")

    print(cm)


    plt.figure(figsize=(7, 5))
```

```
    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=[
            "No Churn",
            "Churn"
        ],
        yticklabels=[
            "No Churn",
            "Churn"
        ]
    )

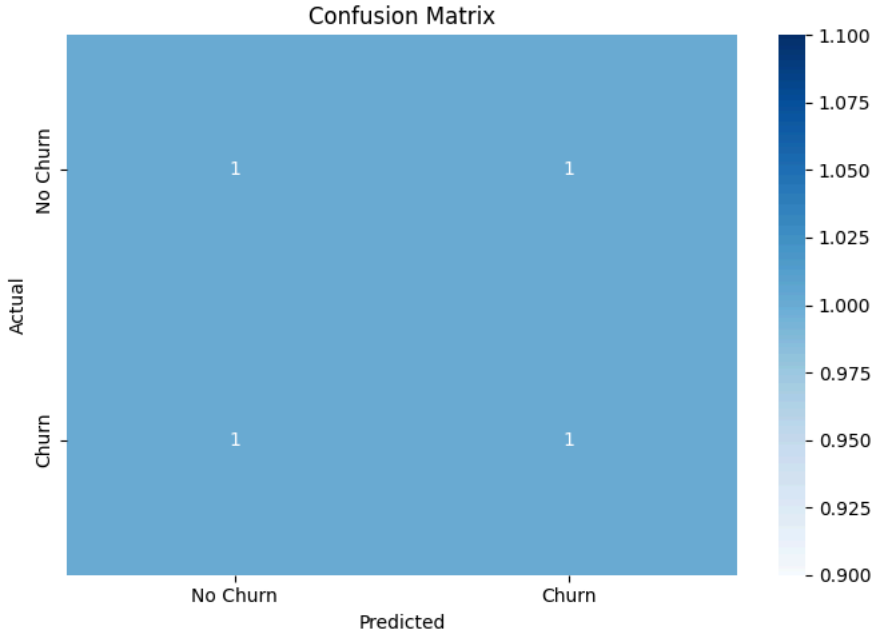

    plt.title(
        "Confusion Matrix"
    )

    plt.xlabel("Predicted")
    plt.ylabel("Actual")

    plt.tight_layout()

    plt.show()
    print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
CONFUSION MATRIX
============================================================
[[1 1]
 [1 1]]
```




```
Himani Chaudhary
CU240250967
```

```
# 24. CLASSIFICATION REPORT
# ============================================================

if len(y_actual) > 0:

    print("\n============================================================")
    print("CLASSIFICATION REPORT")
    print("============================================================")


    print(
        classification_report(
            y_actual,
            y_predicted,
            labels=[0, 1],
            target_names=[
                "No Churn",
                "Churn"
```

```
            ],
            zero_division=0
        )
    )
    print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

    No Churn       0.50      0.50      0.50         2
       Churn       0.50      0.50      0.50         2

    accuracy                           0.50         4
   macro avg       0.50      0.50      0.50         4
weighted avg       0.50      0.50      0.50         4

Himani Chaudhary
CU240250967
```

```
# 25. ROC-AUC SCORE
# ============================================================

if (
    len(y_actual) > 0
    and probability_column is not None
):

    probability_values = pd.to_numeric(
        df.loc[
            y_actual.index,
            probability_column
        ],
        errors="coerce"
    )


    probability_valid = (
        probability_values.notna()
    )


    if (
        probability_valid.sum() > 0
        and y_actual[
            probability_valid
        ].nunique() == 2
    ):

        try:

            auc_score = roc_auc_score(
                y_actual[
                    probability_valid
                ],
                probability_values[
                    probability_valid
                ]
            )


            print(
                "\nROC-AUC Score:",
                round(auc_score, 4)
            )


        except Exception as e:

            print(
                "\nROC-AUC could not be calculated."
            )

            print(e)


    else:

        print(
            "\nROC-AUC cannot be calculated because valid probability values or both classes are not available."
```

```
        )
print("Himani Chaudhary")
print("CU240250967")
```

```
Himani Chaudhary
CU240250967
```

```
# 26. ACTUAL VS PREDICTED COMPARISON
# ============================================================

if (
    actual_numeric is not None
    and predicted_numeric is not None
):

    comparison = pd.DataFrame({

        "Actual": actual_numeric,

        "Predicted": predicted_numeric

    })


    comparison = comparison.dropna()


    if len(comparison) > 0:

        actual_count = comparison[
            "Actual"
        ].sum()


        predicted_count = comparison[
            "Predicted"
        ].sum()


        print("\n============================================================")
        print("ACTUAL VS PREDICTED CHURN")
        print("============================================================")


        print(
            "Actual Churn Customers:",
            int(actual_count)
        )


        print(
            "Predicted Churn Customers:",
            int(predicted_count)
        )


        plt.figure(figsize=(8, 5))


        plt.bar(
            [
                "Actual Churn",
                "Predicted Churn"
            ],
            [
                actual_count,
                predicted_count
            ]
        )

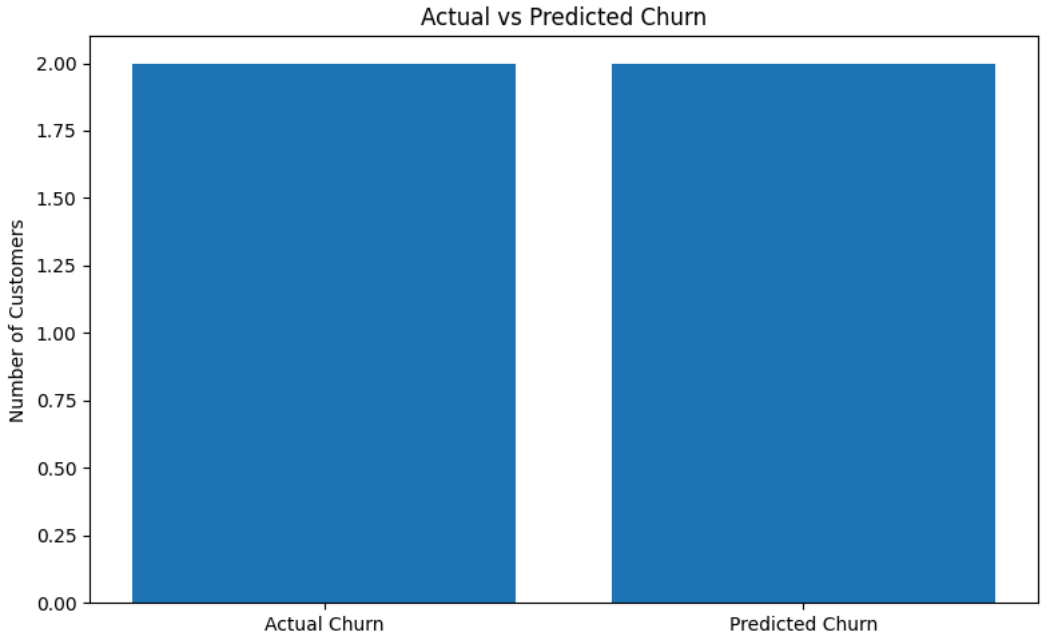

        plt.title(
            "Actual vs Predicted Churn"
        )


        plt.ylabel(
            "Number of Customers"
        )
```

```
        plt.tight_layout()

        plt.show()
        print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
ACTUAL VS PREDICTED CHURN
============================================================
Actual Churn Customers: 2
Predicted Churn Customers: 2
```




```
Himani Chaudhary
CU240250967
```

```
# 27. CORRECT AND INCORRECT PREDICTIONS
# ============================================================

if len(y_actual) > 0:

    correct_predictions = (
        y_actual == y_predicted
    )


    correct_count = (
        correct_predictions.sum()
    )


    incorrect_count = (
        len(correct_predictions)
        -
        correct_count
    )


    print("\n============================================================")
    print("CORRECT VS INCORRECT PREDICTIONS")
    print("============================================================")


    print(
        "Correct Predictions:",
        int(correct_count)
    )


    print(
        "Incorrect Predictions:",
        int(incorrect_count)
    )


    plt.figure(figsize=(8, 5))
```

```
    plt.bar(
        [
            "Correct",
            "Incorrect"
        ],
        [
            correct_count,
            incorrect_count
        ]
    )

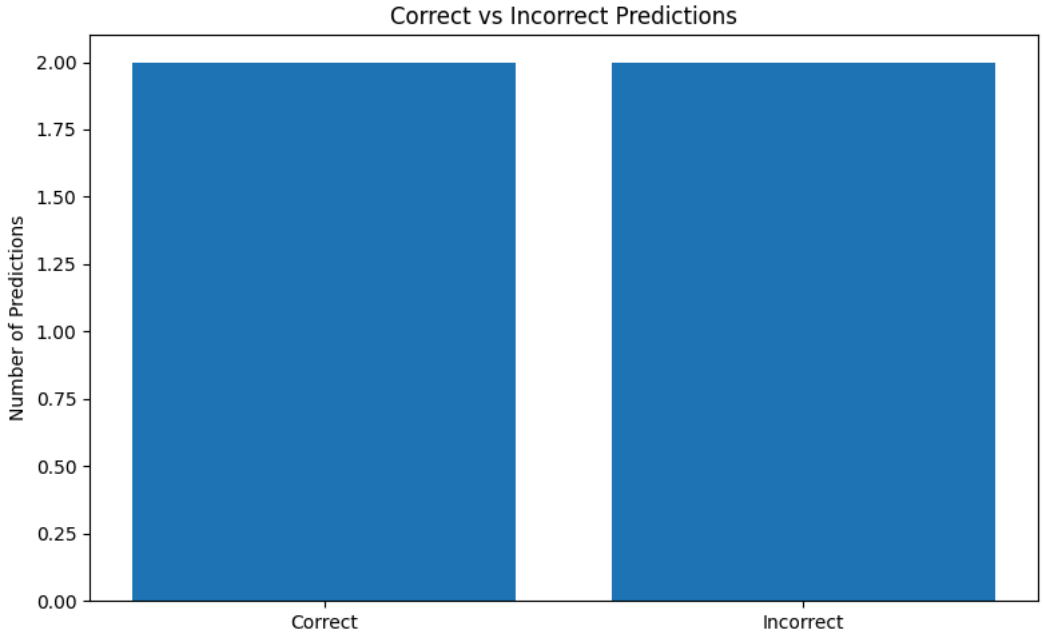

    plt.title(
        "Correct vs Incorrect Predictions"
    )


    plt.ylabel(
        "Number of Predictions"
    )


    plt.tight_layout()

    plt.show()
    print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
CORRECT VS INCORRECT PREDICTIONS
============================================================
Correct Predictions: 2
Incorrect Predictions: 2
```




```
Himani Chaudhary
CU240250967
```

```
# 28. CHURN RATE
# ============================================================

if actual_numeric is not None:

    valid_actual = (
        actual_numeric.dropna()
    )


    if len(valid_actual) > 0:

        churn_count = (
            valid_actual.sum()
        )
```

```
        total_customers = (
            len(valid_actual)
        )


        churn_rate = (
            churn_count
            /
            total_customers
        ) * 100


        print("\n============================================================")
        print("CUSTOMER CHURN RATE")
        print("============================================================")


        print(
            "Total Customers:",
            total_customers
        )


        print(
            "Churned Customers:",
            int(churn_count)
        )


        print(
            "Churn Rate:",
            round(churn_rate, 2),
            "%"
        )
        print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
CUSTOMER CHURN RATE
============================================================
Total Customers: 4
Churned Customers: 2
Churn Rate: 50.0 %
Himani Chaudhary
CU240250967
```

```
# 29. RISK CATEGORIZATION
# ============================================================

if probability_column is not None:

    print("\n============================================================")
    print("CUSTOMER RISK CATEGORIZATION")
    print("============================================================")


    probability_values = pd.to_numeric(
        df[probability_column],
        errors="coerce"
    )


    def risk_category(probability):

        if pd.isna(probability):

            return "Unknown"

        elif probability >= 0.70:

            return "High Risk"

        elif probability >= 0.40:

            return "Medium Risk"

        else:

            return "Low Risk"
```

```
    df["Risk_Category"] = (
        probability_values.apply(
            risk_category
        )
    )


    print(
        df["Risk_Category"].value_counts()
    )


    plt.figure(figsize=(8, 5))


    sns.countplot(
        x=df["Risk_Category"],
        order=[
            "High Risk",
            "Medium Risk",
            "Low Risk",
            "Unknown"
        ]
    )


    plt.title(
        "Customer Churn Risk Categories"
    )


    plt.xlabel(
        "Risk Category"
    )


    plt.ylabel(
        "Number of Customers"
    )


    plt.tight_layout()

    plt.show()


else:

    print(
        "\nRisk categorization skipped because probability column was not found."
    )
    print("Himani Chaudhary")
print("CU240250967")
```

```
Risk categorization skipped because probability column was not found.
Himani Chaudhary
CU240250967
```

```
# 30. HIGH-RISK CUSTOMER ANALYSIS
# ============================================================

if "Risk_Category" in df.columns:

    high_risk_customers = df[
        df["Risk_Category"] == "High Risk"
    ]


    print("\n============================================================")
    print("HIGH-RISK CUSTOMER ANALYSIS")
    print("============================================================")


    print(
        "Number of High-Risk Customers:",
        len(high_risk_customers)
    )


    if len(high_risk_customers) > 0:
```

```
        print(
            "\nHigh-Risk Customer Records:"
        )


        print(
            high_risk_customers.head(10)
        )
print("Himani Chaudhary")
print("CU240250967")
```

```
Himani Chaudhary
CU240250967
```

```
# 31. BUSINESS INSIGHTS
# ============================================================

print("\n============================================================")
print("BUSINESS INSIGHTS")
print("============================================================")


if actual_numeric is not None:

    valid_actual = (
        actual_numeric.dropna()
    )


    if len(valid_actual) > 0:

        churn_rate = (
            valid_actual.mean()
            *
            100
        )


        print(
            "\n1. Overall customer churn rate:",
            round(churn_rate, 2),
            "%"
        )


        if churn_rate >= 30:

            print(
                "   The churn rate is high."
            )


        elif churn_rate >= 15:

            print(
                "   The churn rate is moderate."
            )


        else:

            print(
                "   The churn rate is relatively low."
            )


if probability_column is not None:

    if "Risk_Category" in df.columns:

        high_risk_count = (
            df["Risk_Category"] == "High Risk"
        ).sum()


        print(
            "\n2. High-risk customers:",
            int(high_risk_count)
        )
```

```
        print(
            "   These customers should receive retention offers."
        )


if len(y_actual) > 0:

    accuracy = accuracy_score(
        y_actual,
        y_predicted
    )


    print(
        "\n3. Prediction accuracy:",
        round(accuracy * 100, 2),
        "%"
    )


    if accuracy >= 0.80:

        print(
            "   The prediction model performs well."
        )


    elif accuracy >= 0.60:

        print(
            "   The prediction model has moderate performance."
        )


    else:

        print(
            "   The prediction model needs improvement."
        )
        print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
BUSINESS INSIGHTS
============================================================

1. Overall customer churn rate: 50.0 %
   The churn rate is high.

3. Prediction accuracy: 50.0 %
   The prediction model needs improvement.
Himani Chaudhary
CU240250967
```

```
# 32. BUSINESS RECOMMENDATIONS
# ============================================================

print("\n============================================================")
print("BUSINESS RECOMMENDATIONS")
print("============================================================")


print("""
1. Identify customers with a high probability of churn.

2. Provide special discounts and offers to high-risk customers.

3. Improve customer service and complaint resolution.

4. Contact customers before they decide to leave.

5. Study customer behavior and identify the major reasons
   for customer churn.

6. Develop personalized customer retention campaigns.

7. Regularly monitor the customer churn rate.

8. Use machine learning predictions to support
```

```
   customer retention decisions.
""")
print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
BUSINESS RECOMMENDATIONS
============================================================

1. Identify customers with a high probability of churn.

2. Provide special discounts and offers to high-risk customers.

3. Improve customer service and complaint resolution.

4. Contact customers before they decide to leave.

5. Study customer behavior and identify the major reasons
   for customer churn.

6. Develop personalized customer retention campaigns.

7. Regularly monitor the customer churn rate.

8. Use machine learning predictions to support
   customer retention decisions.

Himani Chaudhary
CU240250967
```

```
# 33. SAVE CLEANED DATASET
# ============================================================

# Save to a DIFFERENT file so that the original dataset
# remains unchanged.

output_file ="/content/churn_prediction_results.csv"


try:

    df.to_csv(
        output_file,
        index=False
    )


    print("\n============================================================")
    print("CLEANED DATASET SAVED")
    print("============================================================")


    print(
        "File saved successfully at:"
    )


    print(
        output_file
    )


except Exception as e:

    print(
        "\nERROR while saving cleaned dataset:"
    )

    print(e)
    print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
CLEANED DATASET SAVED
============================================================
File saved successfully at:
/content/churn_prediction_results.csv
CU240250967
```

```
# 34. FINAL CONCLUSION
# ============================================================

print("\n============================================================")
print("FINAL CONCLUSION")
print("============================================================")


print("""
The customer churn dataset was successfully analyzed using
Python data analytics techniques.

The project included:

- Dataset loading
- Data cleaning
- Missing value handling
- Duplicate value handling
- Descriptive statistics
- Numerical analysis
- Categorical analysis
- Data visualization
- Correlation analysis
- Chi-square testing
- Churn analysis
- Prediction evaluation
- Confusion matrix
- Classification report
- ROC-AUC analysis
- Risk categorization
- High-risk customer analysis
- Business insights
- Business recommendations
- Cleaned dataset generation

The analysis can help a business identify customers who are
likely to leave and take suitable retention actions.
""")
print("Himani Chaudhary")
print("CU240250967")
```

```
============================================================
FINAL CONCLUSION
============================================================

The customer churn dataset was successfully analyzed using
Python data analytics techniques.

The project included:

- Dataset loading
- Data cleaning
- Missing value handling
- Duplicate value handling
- Descriptive statistics
- Numerical analysis
- Categorical analysis
- Data visualization
- Correlation analysis
- Chi-square testing
- Churn analysis
- Prediction evaluation
```